LAB 7: TRANSACTIONS AND ISOLATION LEVELS

Topic 9 — University Database

Student ID: 24125093
Full name: Nguyễn Đình Thiên Lộc

1. Stored-procedure solution

The complete executable definitions are in `24125093.sql`. They target the supplied `University` database and the exact column types in its `Department` and `Instructor` tables. Every procedure uses an explicit transaction, `TRY/CATCH`, `XACT_ABORT`, and rollback on failure. No `SELECT` lock hint is used.

1.1 Department payroll summary

`usp_GetDepartmentPayrollSummary(@DepartmentID)` returns:

Column	Meaning	Implementation
`department_name`	Department name	Grouped department row
`NumberOfInstructors`	Number of instructors	`COUNT(instructor_id)`
`TotalSalary`	Department payroll	`SUM(salary)`, converted to `bigint`

A left join makes an existing department with no instructors return a count and total of zero. An unknown department raises error 50001 and rolls the transaction back. Converting salary to `bigint` prevents integer overflow when a department has many instructors.

1.2 Adjust instructor salary

`usp_AdjustInstructorSalary(@InstructorID, @NewSalary)` issues one atomic `UPDATE` and checks `@@ROWCOUNT`. Exactly one matching instructor is updated and committed. If there is no match, or if the salary is non-positive, the procedure throws an error and rolls back. This avoids a separate "check then update" window.

1.3 Add a new instructor

`usp_AddNewInstructor(@InstructorID, @InstructorName, @Phone, @DepartmentID, @Salary)` attempts the `INSERT` directly. The primary key on `instructor_id` is the atomic existence test: a new ID commits; a duplicate raises SQL Server error 2601/2627, is translated to error 50005, and rolls back. Other constraint failures, including an invalid department or salary, also roll back. This design is safe without a `SELECT` with locks.

2. Concurrency-conflict analysis

Yes, simultaneous executions can conflict. The important cases are:

1. Two salary procedures target the same instructor. Their exclusive row locks cannot be held simultaneously. One waits; after the first commits, the second may overwrite it. If both new salaries were

calculated from the same old salary outside the procedure, this is a lost-update business conflict even though each SQL statement is atomic.

2. Two add procedures use the same instructor ID. One insert wins. The other waits if necessary, then fails the primary-key constraint and rolls back. No duplicate can commit.
3. A payroll summary overlaps a salary update or insert. Its value and its blocking behaviour depend on the isolation level.

2.1 Isolation-level scenarios

Isolation level	Example interleaving	Result and risk
READ UNCOMMITTED	Session 1 changes I001's salary but has not committed; Session 2 calculates the CS payroll.	Session 2 may include the uncommitted salary (dirty read), and can report a value that never becomes real if Session 1 rolls back. It can also observe unstable rows. Write/write conflicts still take exclusive locks.
READ COMMITTED	Session 1 updates I001 while Session 2 requests the CS payroll.	Dirty reads are prevented. With normal locking, the summary can wait for Session 1. Each statement sees committed data, but repeated statements/calls may see a changed salary or a newly inserted instructor (non-repeatable read or phantom).
REPEATABLE READ	Session 2 reads the CS instructors twice in one transaction while Session 1 changes an existing instructor and Session 3 inserts a new CS instructor.	Existing rows read by Session 2 cannot change until it ends, so the update waits. A new matching row may still appear: a phantom. Longer shared locks increase blocking and deadlock risk.
SERIALIZABLE	Session 2 reads the CS payroll while Session 1 attempts to insert another CS instructor.	Key-range protection prevents the phantom, so the insert/read must wait or a deadlock victim is chosen. The result is serializable but concurrency is lowest.
SNAPSHOT	Session 2 starts a snapshot transaction, then Session 1 commits a salary change or new instructor before Session 2 reads again.	Session 2 keeps its transaction-start view, so payroll is stable and readers do not block writers. Two snapshot transactions updating the same instructor cannot both commit; the later one can fail with update-conflict error 3960 and must be retried.

3. Proposed solution

The submitted implementation combines the following controls:

- Keep transactions short and use atomic DML. The salary procedure uses one conditional `UPDATE`; the add procedure relies on the primary key instead of an unsafe pre-check.
- Preserve database constraints. The instructor primary key guarantees uniqueness, the department foreign key prevents orphan instructors, and the salary check prevents invalid pay.
- Use `TRY/CATCH`, `XACT_ABORT`, and `XACT_STATE()` so every failed unit of work is rolled back and no transaction is left open.
- For reporting workloads, enable `READ_COMMITTED_SNAPSHOT` at database-administration level and keep the summary at READ COMMITTED. This gives a consistent statement-level payroll without

reader/writer blocking. For one historical view shared by several report statements, enable the database's snapshot-isolation option and run all statements in one SNAPSHOT transaction.

- Retry transient deadlock error 1205 and snapshot update-conflict error 3960 in the caller, using a short bounded retry with backoff.

If the application calculates a new salary from a previously displayed salary, add optimistic concurrency to prevent a stale overwrite. For example, accept `@ExpectedSalary` (or a `rowversion`) and update with both `instructor_id = @InstructorID` and `salary = @ExpectedSalary`. A zero-row result then means the row changed and the caller must reload it. This preserves the assignment's no-`SELECT`-with-locks restriction.

4. Conclusion

The procedures satisfy commit/rollback requirements and are race-safe for duplicate insertion. Isolation level still determines the payroll view, blocking, and update-conflict behaviour. Atomic writes plus constraints and row-versioned reads provide the best balance for this workload; optimistic version checking is needed when a stale salary must never overwrite a newer one.